It's not easy to hit the sweet spot between too much eval and no eval at all. If you don't want to abstain from eval completely, Ruby does provide some features that make it somewhat safer. Let's take a look at them.

Tainted Objects and Safe Levels

Ruby automatically marks potentially unsafe objects—in particular, objects that come from external sources—as *tainted*. Tainted objects include strings that your program reads from web forms, files, the command line, or even a system variable. Every time you create a new string by manipulating tainted strings, the result is itself tainted. Here's an example program that checks whether an object is tainted by calling its tainted? method:

ctwc/tainted_objects.rb

```
# read user input
user_input = "User input: #{gets()}"
puts user_input.tainted?
```

```
⇒ x = 1
❮ true
```

If you had to check every string for taintedness, then you wouldn't be in a much better position than if you had simply tracked unsafe strings on your own. But Ruby also provides the notion of *safe levels*, which complement tainted objects nicely. When you set a safe level (which you can do by assigning a value to the $SAFE global variable), you disallow certain potentially dangerous operations.

You can choose from four safe levels, from the default 0 ("hippie commune," where you can hug trees and format hard disks) to 3 ("military dictatorship," where every object you create is tainted by default). A safe level of 2, for example, disallows most file-related operations. Any safe level greater than 0 also causes Ruby to flat-out refuse to evaluate tainted strings:

```
$SAFE = 1
user_input = "User input: #{gets()}"
eval user_input
```

```
⇒ x = 1
❮ SecurityError: Insecure operation - eval
```

Ruby 2.0 and earlier also had a safe level of 4 that didn't even allow you to exit the program freely. For various reasons, this extreme safe level turned out to be not as secure as people assumed it would be, so it has been removed in Ruby 2.1.